

1. Introduction

I designed a cybersecurity plan for a hypothetical smart factory that uses an AI-integrated IIoT system for predictive maintenance and basic quality monitoring. The goal was to map out what the system looks like, find realistic vulnerabilities, design defenses that a small team could actually implement, and then think through some simple penetration-testing style simulations.

2. System Description

2.1 System overview

The system is a smart factory machine-health monitoring setup. Machines on the factory floor have sensors (temperature, vibration, current) feeding data into an edge device, which then sends the data to the cloud, where AI models predict potential failures and flag anomalies.

The main goals are:

- Spot failing machines early (predictive maintenance).
- Give operators a dashboard to see machine health.
- Use AI to highlight unusual behavior instead of relying only on manual checks.

2.2 Components

1. Devices and sensors
 - Vibration, temperature, and current sensors on motors/gearboxes.
 - A small PLC or microcontroller that collects sensor data.
2. Edge device/gateway
 - An industrial PC or Raspberry Pi-style device that receives data from the PLC, does some preprocessing, and publishes data using MQTT.
3. Network
 - Local OT network on the factory floor.
 - MQTT broker on the local network.
 - A firewall/router that connects the local network to the internet and then to a cloud IoT platform.
4. Cloud and apps
 - Cloud IoT hub that ingests data.
 - Time-series database to store sensor readings.
 - A simple web dashboard so operators can log in and see graphs and alerts.
5. AI models

- A basic predictive model to estimate if a machine is likely to fail soon.
 - A simple anomaly detection model that flags out-of-range values or weird patterns.
6. Users
- Operators: check dashboards and respond to alerts.
 - Maintenance engineers: schedule repairs and look at detailed data.
 - Admins/IT: manage users, devices, network, and cloud access.

2.3 Data flow

1. Sensors collect readings every few seconds and send them to the PLC.
2. The PLC sends data to the edge device (for example, via Modbus/TCP).
3. The edge device pre-processes the data, runs a local AI model for quick checks, and then publishes the data to an MQTT broker.
4. The MQTT broker forwards data through a secure connection to a cloud IoT hub.
5. Data is stored in a time-series database, and AI models in the cloud get trained and updated using the historical data.
6. The web dashboard uses APIs to fetch data and AI predictions and displays them to users.

3. Vulnerability Assessment

I split the vulnerabilities by layer so it's easier to see where issues can show up. This is based on common IIoT/IoT security problems students and industry projects run into.

3.1 Devices and firmware

1. Default or weak passwords on PLCs/edge devices
 - Example: using “admin/admin” or vendor defaults. An attacker on the network could log in and change settings or upload malicious code.
2. Outdated or unsigned firmware
 - If firmware updates aren't signed or checked, someone could push a modified firmware image and get persistent access.
3. Unnecessary open services
 - Things like Telnet or an unencrypted HTTP admin page left enabled can be easy entry points if exposed.

3.2 Network and protocols

- MQTT without proper TLS

- If MQTT runs in cleartext or with bad certificate validation, attackers can sniff or modify messages (like faking sensor data or alerts).
- Flat network with no segmentation
 - If the factory floor network and office network are basically one big flat network, compromising a single laptop can give a path into the OT side.
- Weak VPN/remote access controls
 - If remote access is allowed with simple passwords and no MFA, stolen credentials can give broad access.

3.3 Data and storage

1. Unencrypted databases and storage
 - If the time-series database or cloud storage is not encrypted, someone who gets access to the DB or bucket can read or modify all the historical data.
2. Over-permissive cloud roles
 - Misconfigured IAM roles can give users or services more access than they actually need.
3. Minimal logging on data access
 - If data reads and writes aren't properly logged, it's hard to notice slow data theft or tampering.

3.4 Application and API

1. Weak or missing input validation on the web dashboard
 - For example, parameters passed into queries without proper sanitization can open the door to injection attacks.
2. Poor session management
 - Long-lived sessions, no logout, or insecure cookies can make session hijacking easier.
3. APIs without strong auth or rate limiting
 - If APIs are open or only lightly protected, attackers can brute force credentials or hammer endpoints for data.

3.5 AI models and data pipeline

1. Data poisoning (training data tampered with)
 - If historical sensor data can be changed, someone could inject fake but “valid-looking” readings so the model learns wrong patterns, like treating early failure signs as normal.
2. No monitoring for model drift

- If nobody is watching how model performance changes over time, slow drift or poisoning can go unnoticed.
3. Overexposed model APIs
 - If models are accessible over an API without limits, an attacker could spam queries to reverse engineer the model behavior.

3.6 Human and physical factors

1. Phishing and social engineering
 - Operators and admins can be tricked into giving up passwords or approving unexpected remote sessions.
2. Weak password habits and no MFA
 - Reusing or using simple passwords across systems makes credential stuffing much easier.
3. Poor physical security for field devices
 - Edge devices sitting in open areas with accessible ports or SD cards are very easy to tamper with if someone gets on-site.

4. Defense Strategy

The plan is basically “defense in depth” but scaled down so it feels realistic for a student project: focus on core hardening, basic best practices, and a few AI-specific protections.

4.1 Devices and firmware

- Change all default passwords and use unique, strong passwords per device; store them in a password manager instead of a spreadsheet.
- Enable secure boot and signed firmware where supported, and keep a simple firmware update process with checksums or signatures.
- Disable unused services and ports (e.g., turn off Telnet and plain HTTP, use SSH and HTTPS with proper configs).

4.2 Authentication and access control

- Use role-based access control (RBAC) in the dashboard and cloud platform so operators, engineers, and admins have clearly different permissions.
- Turn on MFA for admin accounts and remote access accounts; this is realistically doable with most cloud platforms and identity providers.
- Give devices their own identities (certificates or unique credentials) and require mutual TLS between edge devices and the MQTT broker/cloud.

4.3 Network and transport security

- Use TLS for MQTT and HTTPS everywhere, with proper certificate validation.
- Split the network into at least two zones: OT (factory devices) and IT (office / general use), using VLANs and a firewall between them.
- Limit inbound connections and use a VPN for remote access instead of exposing services directly.

4.4 Data and storage protection

- Enable encryption at rest in the time-series database and storage buckets using built-in cloud options.
- Lock down IAM roles to least privilege, and avoid using overly broad “admin” roles for normal services.
- Turn on logging for data access so reads, writes, and permission changes are tracked.

4.5 Application and API security

- Apply basic secure coding practices: input validation, parameterized queries, and not trusting user input.
- Protect APIs with proper authentication (tokens/OAuth) and add rate limits to stop brute forcing.
- Tighten session management (shorter token lifetimes, secure cookies, CSRF protection) on the dashboard.

4.6 AI and data-pipeline defenses

- Lock down who can write to the training data store, and run basic validation checks on data before it’s used for training (ranges, expected formats, etc.).
- Track simple metrics like model accuracy over time or alert rate changes; big unexplained shifts are a red flag.
- Put access controls and rate limits on any model inference APIs so they can’t be easily abused or scraped.

4.7 Monitoring and incident response

- Centralize logs (device logs, application logs, and cloud logs) into one place and configure some straightforward alerts (e.g., multiple failed logins, new device registrations, unusual traffic).
- Write short, practical incident response steps for a few key scenarios: suspicious login, MQTT issues, unexpected data spikes, or suspected data poisoning.

- Run quick “tabletop” simulations where you walk through what you’d do if an alert fired, instead of just having a document nobody reads.

4.8 Physical and user-focused security

- Put edge devices and network equipment in locked boxes or rooms where possible, and use tamper-evident seals if devices are exposed.
- Give short security awareness sessions to operators and engineers on phishing, password hygiene, and how to report weird behavior.
- Enforce a simple password policy and encourage using a password manager instead of writing passwords on sticky notes.

5. Implementation Plan

The scope is realistic: design configs, small proofs-of-concept, and written procedures, and show how it would work.

5.1 Timeline

Week	Focus	What actually gets done
1	System mapping & threat modeling	Draw system diagram, list assets, and identify vulnerabilities for each layer.
2	Core hardening plan	Define password/MFA policies, device hardening steps, basic network segmentation, and TLS decisions.
3	App, AI, and data protections	Draft rules for API security, training data validation, and simple model monitoring checks.

4	“Pen test” simulations & wrap-up	Run small attack simulations in a lab or virtual environment, write up what worked, what failed, and what improvements.
---	----------------------------------	---

Even if it can’t deploy everything physically, it can still show:

- Sample configuration snippets (e.g., MQTT with TLS setup).
- Example access control / IAM policies.
- Pseudo-logs or screenshots from test runs to illustrate detection and response.

6. Penetration-Testing Style Simulations

Instead of full enterprise-level pen-testing, this is modeled as simple red-team vs blue-team exercises that can be done with a few VMs, containers, or even simulated endpoints.

6.1 Scenarios

1. MQTT / network attack (MITM or message injection)
 - Red team: tries to sniff unencrypted MQTT traffic or inject fake sensor messages in a test environment.
 - Blue team: responds by enabling TLS, checking certificate validation, and verifying that fake messages no longer get through.
2. Web/API attack
 - Red team: runs a basic scanner or manual tests against the demo dashboard/API to look for weak authentication or missing input validation.
 - Blue team: tightens input validation, rate limiting, and authentication, then shows that the same attack attempt now fails.
3. AI data poisoning test (small-scale)
 - Red team: modifies a small subset of the training CSV (or dataset) in a controlled lab environment to flip labels or insert weird values.
 - Blue team: introduces simple data validation checks and monitors the model metrics before and after to show the impact and how to detect it.
4. Social engineering / credential test (simulated)
 - Red team: creates a fake “phishing” message as a demo, or simulates someone asking for credentials in a scenario.
 - Blue team: shows how awareness training and MFA reduce the impact, and writes clear reporting steps.

6.2 Metrics

For each test, it tracks:

- Whether the attack succeeded the first time.
- What changed after defenses were applied (e.g., TLS, rate limiting, validation)
- How quickly would the issue be noticed based on logs and alerts?

7. Assessment and Improvements

7.1 What worked well

- Just mapping the system and doing threat modeling made it much easier to identify obvious problems, such as default passwords and a lack of segmentation.
- Basic fixes like enabling TLS for MQTT and adding MFA for admin accounts provided noticeable security improvements for relatively low effort.
- Even simple checks on training data (e.g., ranges, consistency) helped demonstrate how data poisoning can be detected or at least made harder.

7.2 What still needs work

- Full device-level security (secure boot, signed firmware) depends heavily on hardware support and vendor tools, which are harder to replicate in a student project.
- AI security is more than just data validation; real systems would need more advanced monitoring, adversarial training, and better tooling than a basic student setup provides.
- Human factors (like culture and training) can't be "fixed" in four weeks; they need ongoing effort.

8. Personal Reflection

Working on this project made me realize that securing an IIoT system is less about one fancy tool and more about a lot of small, boring decisions done consistently (like password policies, segmenting networks, and turning on TLS). I also saw how adding AI into the mix doesn't magically solve security problems, it actually creates new ones, especially around training data and model behavior.

From a practical standpoint, the parts that felt most doable as a student were the architecture diagrams, threat modeling, and basic hardening steps like changing defaults, planning network segmentation, and writing simple incident response playbooks. The more

advanced features (secure boot, full SIEM setup, sophisticated AI defenses) are probably out of scope to fully implement this project, but they can still be described and prototyped at a small scale.